

CS 474: BirdNet Final Project

Problem

In this paper, I explore using several different deep learning architectures and techniques to improve bird song classification accuracy. This is a supervised task since the models will be provided with labeled examples of bird songs. I am not personally an avid birder, so I do not have any domain expertise to apply to the problem. Perhaps the most common approach to bird song classification involves using CNNs to analyze audio spectrograms. I replicated that approach and also tried using 1D CNNs and Transformers to analyze raw audio.

Exploratory Data Analysis

I used audio from a collection of fully-annotated soundscape recordings from the Western United States published by Cornell Lab of Ornithology [here](#) for the 2021 BirdCLEF bird call identification competition. The dataset includes 33 hour-long soundscape recordings, which have been annotated with 20,147 bounding box labels for 56 different bird species.

I noticed quite a few super short annotations, so I filtered the data to only include segments longer than 2 seconds. I also only used data from species with more than 100 recordings each since some species were very underrepresented. After filtering, I was left with about 12,500 annotations for 17 species. I then created an individual audio file and spectrogram for each annotation segment. Each audio segment was normalized to have a mean of 0 and variance of 1 so that models are less sensitive to volume. Upon inspecting the spectrograms, I saw clear patterns that could distinguish each species' song. This gave me confidence the task was feasible.

Methodology

The purpose of this experiment is to see the effect of different input formats, model architectures, and model sizes on classification accuracy. I chose two input formats (raw audio and spectrograms), three types of model architecture (Transformers, 2D CNNs, and 1D CNNs), and various model sizes. For larger models, I used pretrained weights and finetuned.

For Transformers, I chose HuBERT and WavLM since they are both popular open-source audio models that have been pretrained on many hours of human speech data. For 2D CNNs, I chose VGG-11 and YAMNet. VGG-11 was pretrained on data for the 1,000-class ImageNet competition. I replaced the head to classify only 17 classes instead of the full 1,000. Given the large model size and broad training data, it likely has many useful features to extract. YAMNet was designed specifically for audio classification, but I did not use its weights since they were saved in a TensorFlow format. I implemented the basic architecture in PyTorch instead. Unlike VGG-11, YAMNet uses both point-wise and depth-wise separable convolutions.

I designed my own 1D CNN for raw audio classification since we did not have experience with 1D CNNs in class. It contains 3 convolutional layers with large kernels (starting at 80, which corresponds to about 5ms of audio sampled at 16kHz) and an increasing number of output channels with each layer ($32 \rightarrow 64 \rightarrow 128$) as seems to be typical in most CNNs. These feed into an adaptive average pooling layer followed by two linear layers of dimension 64 and 17 (the final number of output classes). All models use cross-entropy loss in training, so softmax is not necessary. This first model did not perform super well, so I tried more convolutional layers of larger size ($64 \rightarrow 128 \rightarrow 256 \rightarrow 512 \rightarrow 512 \rightarrow 512$) to increase the receptive field. This second model is known as "1D CNN v2."

I ended up paring down my dataset to just 2,000 training examples and 500 test examples (an 80/20 split) for quicker training and experimentation. I made sure that both the training and test set have the same distribution of target classes. Unfortunately, I did not think to check if there were any overlapping audio clips until writing this report. About 25% of the audio clips overlap each other.

For training, I used the Adam optimization algorithm with weight decay for regularization. Both weight decay and the learning rate were set to $1e-5$. I tried a higher learning rate at first, but it ended up being way too fast. A high learning rate caused pretrained models like VGG-11 to lose their pretrained knowledge and converge to just selecting the majority class. I had also tried freezing weights and removing layers, but this did not help.

Results and Discussion

The table to the right contains the parameter counts and test set scores for each of the models explained above. Also included at the bottom is the baseline accuracy (always choosing the majority class) and the performance of both VGG-11 and YAMNet when given all of the prepared training samples (around 10,000 spectrograms). While 75% is a modest accuracy, it is actually pretty impressive when you consider that around 25% of the audio clips overlap. That means that multiple bird calls were present in the same spectrogram.

Model	Parameters	Macro F1	Accuracy
<i>Transformers (Raw Audio)</i>			
Ft HuBERT	24M	0.549	0.708
Ft WavLM	94M	0.548	0.696
<i>2D CNN (Spectrogram)</i>			
Ft VGG-11	129M	0.562	0.748
YAMNet	3M	0.239	0.532
<i>1D CNN (Raw Audio)</i>			
1D CNN v1	43K	0.112	0.366
1D CNN v2	5M	0.139	0.334
<i>Baseline</i>			
Majority Class	0	0.020	0.203
<i>More training data</i>			
Ft VGG-11	129M	0.659	0.816
YAMNet	3M	0.297	0.571

Input Representation: I expected spectrograms to be more rich representations of the audio and thus perform better. This turned out to be true, although I was surprised with how well Transformers were able to perform with just raw audio.

Model size: The largest model (VGG-11) did perform the best. However, smaller models like HuBERT were able to approach its superior performance. Even smaller models like YAMNet were able to punch well above their weight with some clever architectural design.

Pretrained weights: Pretrained models certainly helped to boost accuracy. Finetuned models performed the best, and I would not have been able to train models of that size to a good accuracy level given how little training data I had without pretraining.

Architecture: As mentioned, YAMNet's unique architecture seemed to have a positive impact. It achieved moderate results even without pretraining. Its much larger 2D CNN counterpart VGG-11 achieved the high score. My larger 1D CNN started to overfit after a couple epochs. This was apparent due to the test accuracy taking a sharp dip as training progressed. It was probably overparameterized. Transformers far outperform 1D CNNs with raw audio.

Conclusion

My comparison of models reveals that 2D CNNs with spectrograms remain state-of-the-art for bird song classification. Pretraining is powerful—even with seemingly unrelated tasks like image or human speech recognition. An efficient architecture can save a lot of parameters without sacrificing too much performance.